Silent by Default: How Memory Architecture and Runtime Policy Shape Local LLM Agents on AI PCs

Anonymous Author(s)
Submission under double-blind review

***Abstract*—When a local agent's conversation outgrows the context window, runtimes such as Ollama and llama.cpp trim it and still report success. We measure what is lost with pre-registered experiments on an AMD APU, an NVIDIA laptop GPU and an Intel laptop. At windows of 16384 tokens and below, no session of three 8B to 14B models kept its full history, and models up to 70B lose history once the window is exceeded. Rule loss appears in the 8B and 14B models (5 seeds) and in qwen3:32b (2 of 3 sessions at 4096), and not in llama3.3:70b or mistral-small3.2:24b (0 of 3 each). History loss appears in all of them. On a tool-using task workload, qwen3:14b's wrong-task rate rises from 11.3% at its default window to 42.0% at 4096, and every wrong task is silent. Runtime defaults decide how often this happens: an unmodified install loaded a 4096-token window on an Intel CPU and 32768 on the same laptop's integrated GPU. The backend matters too: with identical model files and settings, llama3.1:8b wrote tool calls as text at an untruncated window on Vulkan and CUDA but not on ROCm. The fixes are partial. Setting the window per device removes the hidden default; KV-cache quantization cut session time by 65% to 85% with no measured loss of rules or recall; client-side trimming removes the in-context examples a model imitates; and `num_keep` protects the system prompt only where the runtime shifts or cuts tokens.**

I. Introduction

Runtimes such as Ollama and llama.cpp serve open-weight models of 8B to 70B parameters on laptops and small APU workstations, and local agents drive them for tens of turns, appending every tool result to a conversation that only grows inside a fixed context window. When the conversation outgrows it, the runtime, not the agent, decides what to remove, and it does so silently: the request still returns HTTP 200, the answer is well formed, and the model does not mention the loss (0/960 self-reports in our probes mentioned truncation).

Why run locally, and what it costs. Agent tasks are expensive in the cloud because every step re-sends the whole conversation. Priced on public agent traces (not executed, with token counts recalibrated to the cloud tokenizer), a median task of a long-horizon coding-agent set costs USD 2.5307 on a mid-tier cloud model and USD 0.1354 on a small one with realistic caching. Our own tool-using tasks are short: run fully in the cloud, the small and mid-tier models were correct on 87/90 and 90/90 tasks at a mean USD 0.00067 and USD 0.01020 per task. Running the same tasks locally on qwen3:14b at its default window removes the bill but not the risk: 80/90 tasks were correct, and nothing marked the 10 that were not. The one signal a local runtime could raise, that the prompt exceeded the loaded window, does not find them. At the default window it missed all 17/17 wrong tasks; at 4096 it caught every wrong task only by flagging 147 of 150 tasks, at precision 42.9%. Window overflow is a symptom of one failure mode, not a general wrong-answer detector. A perfect detector would reach 90/90 by escalating 10 tasks, but that is a ceiling, and no such detector exists; fixing a wrong answer afterwards in the cloud fixed 167/197 (84.8%) at a mean of USD 0.0227 per task, but it cannot undo actions already executed (Section VII). The trade-off a practitioner faces is therefore between a cloud bill and silent wrong answers, and how many wrong answers depends on settings the user rarely sees.

A motivating case shows how easily a user lands in this regime. On an Intel laptop, an unmodified Ollama install served two models with a 4096 token window, because device detection drops the integrated GPU and falls back to the CPU. Every agent session crossed that window at turn [3] and then failed silently, 5/5 sessions for each model. With the integrated GPU enabled, the same machine, runtime and models loaded 32768 tokens and no failure came after the window. Nobody chose 4096; the runtime did.

This paper asks what a local agent loses when its conversation exceeds the context window, whether anything tells the user, and what decides how often it happens. We answer with pre-registered experiments on an AMD APU, an NVIDIA laptop GPU and an Intel laptop (CPU and integrated GPU), spanning unified and discrete memory. The answer has five parts. First, history loss is silent and appears in every model we tested, from 8B to 70B: at a 16384-token window no session of three 8B to 14B models kept its full history to turn 15 (intact 5/5/0/0/0/0/0/0 of 5 for each model). Second, rule loss is model-dependent. Rule loss appears in the 8B and 14B models (5 seeds) and in qwen3:32b (2 of 3 sessions at 4096), and not in llama3.3:70b or mistral-small3.2:24b (0 of 3 each). History loss appears in all of them. We do not claim rule loss in general. Third, the cost is concrete: qwen3:14b's share of tasks ending in a wrong state rises from 11.3% at the default window to 42.0% at 4096, every wrong task silent. Fourth, the runtime's defaults and backend decide how often this happens: the window follows the detected device, and with identical model files, settings and seed, llama3.1:8b wrote a tool call as text in place of its answer at an untruncated window on CUDA and on Vulkan, and not on ROCm. Fifth, the fixes are partial. A KV-cache quantization setting cut session time by 85% and

65% for two models with no measured loss of rules or recall, on the platform where the default window is the model's native one; client-side trimming removes the in-context examples the model imitates; and `num_keep` protects the system prompt only where the runtime shifts or cuts tokens.

We make four contributions.

1) **A cross-vendor map of where memory runs out and how it fails** (Section IV). On the Intel integrated GPU a model start succeeds or fails at one shared-heap limit that also counts host-visible buffers, and a first-principles accounting of weights, KV cache and compute predicts the measured flip point to within -257 to -26 tokens.
2) **Runtime policy, not hardware, sets the effective window, and overflow is silent** (Section V). The same runtime gives llama3.1:8b a default window of 4096 tokens on the Intel laptop and 131072 on the AMD APU, and answers an over-length prompt with success after keeping num_ctx/2 + 2 tokens.
3) **The consequences for agents, with their mechanism** (Section VI). Silent failures after the window grow as the window shrinks; Ollama drops whole old messages but keeps the system prompt, and at the smallest window llama.cpp's context shift and a token cut discard it.
4) **Pre-registered fixes and a predictor, with their limits** (Section VII). Client-side trimming is refuted as a general fix on two vendors; `num_keep` lowers qwen3:14b's wrong-task rate at 4096 to 30.0% without removing the excess; a router to a cloud model is refuted on the AMD platform, where a quantized KV cache was on the frontier instead, with the constrained platforms pending; and a context-shift rule taken from the llama.cpp source predicts every observed shift in the sessions it was checked on (misses 0).

Section II covers background, Section III the platforms and harness, Sections IV to VII the results, Section VIII threats to validity, and Section IX concludes.

II. Background and Related Work

Context windows in local runtimes. llama.cpp allocates a KV cache for a fixed number of tokens, n_ctx, when a model loads. When a sequence would grow past it, the server can perform a *context shift*: it keeps the first n_keep tokens, discards a block of the tokens after them (2045 at a 4096 window in our logs) and continues. Ollama serves models through a bundled llama.cpp server and adds two layers. Its chat endpoint renders the message list with the model's template and, when the result exceeds the window, drops whole messages from the front while keeping the system message. It also starts the server with context shift enabled and a keep of 4 tokens, so an over-length prompt succeeds after keeping num_ctx/2 + 2 tokens. The request option `num_keep` sets the kept prefix, which matters only on the shift and cut paths. If the client sends no `num_ctx`, Ollama chooses the window from the detected device: 4096 tokens for llama3.1:8b on our Intel laptop's CPU, 131072 on the AMD APU. A per-request window does not survive a restart, while a model file or environment setting does. None of these steps is reported to the client.

Long-context behaviour. Lost in the Middle [1] shows that models use information in the middle of a long input worse than at its ends; RULER [2] and LongBench [3] find effective context shorter than the advertised window; and multi-turn [4] and rule-following benchmarks [5] [6] measure how well a model uses or obeys what it is given. All assume the model receives the whole input. We study the step before: what the runtime removes, whether anyone can tell, and which models keep their rules when it happens.

KV-cache management. StreamingLLM [7] keeps initial attention-sink tokens plus a recent window, much as a context shift keeps n_keep tokens; H2O [8] and SnapKV [9] evict by attention; KIVI [10] and KVQuant [11] quantize the cache; LLMLingua [12] compresses the prompt. These are designed policies evaluated at fixed memory. We measure what shipped runtimes discard by default, and we treat the runtime's built-in KV quantization as one context policy among several, judged on session time as well as rules and recall (Section VII).

Agents and tool use. AgentBench [13], tau-bench [14], function-calling evaluations [15] and SWE-bench [16] score task success, usually with hosted models and large windows, where running out of context is at most an exit condition. Our task workload scores end state in the same way, but varies the window under a fixed model, runtime and seed.

Routing, cascades and verification. Routers [17] [18] send each query to a small or a large model from a learned estimate of difficulty, and cascades [19] escalate when a scorer judges the cheap answer unreliable. Self-verification [20] and sampling-consistency checks [21] let a model test its own answer. All of these need a signal that correlates with a wrong answer, and none uses the local runtime's effective context. We measure the one such signal a runtime could expose, prompt over the loaded window, and find it blind to wrong answers produced inside the window (Section I), so a hybrid local and cloud deployment needs a general verifier, not a window flag.

Inference on edge hardware. PagedAttention [22] manages KV memory on servers; MELT [23], PowerInfer [24] and LLM in a flash [25] characterize or accelerate inference on phones, consumer GPUs and flash-backed devices in terms of throughput, memory and energy. We add a cross-vendor characterization of behavioural correctness under each runtime's memory and context policy, including how the GPU backend alone changes an agent's output.

III. Platforms, Harness and Methodology

Platforms and runtimes. We study three consumer platforms from three GPU vendors: an AMD Ryzen AI Max+ (Strix Halo) APU whose Radeon iGPU shares unified memory with the CPU; a laptop with an NVIDIA RTX 4070 Laptop GPU and 8188 MiB of discrete VRAM; and an Intel Core Ultra (Panther Lake) laptop with an Arc iGPU on unified memory. Ollama's device detection drops the Intel iGPU and falls back to the CPU, so we measure Intel twice: on the CPU, as shipped, and on the iGPU, enabled only in the environment

of the job's own server process. All agent runs use Ollama, which serves models through a bundled llama.cpp server; the AMD and NVIDIA runs share one pinned release and the model files. Versions and drivers are listed in Table I.

The R2 agent harness. A session is a seeded, deterministic agent conversation of 40 turns. The system prompt carries five checkable rules (JSON answer with fixed keys, call a `log_event` tool, never print a forbidden token, lengths in metres, end with a session code), a two-tool schema and recall facts. Each turn is two calls: on the first the model emits tool calls and the harness simulates results with a long filler payload that grows the context as real tool output does; the second asks for the final answer. Temperature is 0. Canaries in the system prompt and history tags in the first user message, which a message-level trim drops first, detect lost context; each canary turn asks for a new pair, because in our first validation a model copied a reused canary from its own earlier answer and hid a real truncation (qwen3:14b's positive control then detected no truncation turn). A miss counts as truncation only while the final prompt exceeds the context Ollama reports as loaded (`/api/ps`, read every turn), never by the runtime's prompt count.

Call-2 protocol. With tools offered on the second call, llama3.1:8b often answered with a tool call and no text (empty final answers 20); with tools withheld there, it gave none (empty final answers 0). All R2 runs below withhold tools on the final call (the v1 protocol); Section VI repeats the main contrast with tools on every call.

Validation gates. Before any real run, each model passes a validation at a window where nothing is truncated, with gates fixed in advance. A rule is *in use* for a model only if the model follows it on at least 90% of baseline turns; rules below the bar are dropped, not repaired, because the failures are real behaviour. llama3.1:8b logged events on 60.0% of turns and added the session code on 60.0%, so only rules 1, 3 and 4 are in use for it; qwen3:14b kept all five. The same bar decides whether tool validity, arguments and recall are gated. The negative control must miss no canary (0/12 for both models); the positive control, at a small window, must detect truncation (turn [10]); and the task tool must be called correctly on at least 90% of turns, which refused qwen3-4b-2507 at 5/30. A real job refuses to start unless its model's validation passes.

Gated silent failure. For each session we take the first turn on which a gated metric fails. The failure is *silent* when no HTTP error or error field appeared on any earlier turn, and *after-window* when it falls at or after the first turn whose prompt exceeded the loaded window; otherwise it is *baseline-type*, the model's own error rate rather than context loss. The window effect is the *excess over default*: a tier's after-window rate minus the Ollama default tier's, with a 95% percentile bootstrap CI over sessions (10000 resamples, stratified by tier and model). A 0/n or n/n cell has no bootstrap variance, so its CI understates uncertainty at five seeds per cell; one cell was later extended to 10 seeds by its own pre-registration.

Pre-registration and provenance. The R2 kill criterion was written before any session ran: the claim fails if, in every arm, the first failure comes only after an explicit error has surfaced in the same session. The gated definition came later, as analysis, and is reported beside the criterion, never in its place. Every follow-up below was pre-registered the same way, with hypothesis, verdict rule and validity conditions committed before its code was deployed, and we report verdicts in that wording, refutations and inconclusive outcomes included. Every number is pasted from a numbers register that one script regenerates from the raw result files; a claim with no verified row is not made. One provenance gap remains: runs before 2026-10-14 did not record the harness file's blob, so their harness identity rests on byte-identical outputs across reruns (identical final text 120/120). Rows are never pooled across platforms, and a configuration flag counts only when the runtime's log shows it applied: an earlier sweep passed a KV-cache precision setting that the runtime silently ignored.

TABLE I
PLATFORMS. CONFIGURATION RECORD, NOT A MEASURED RESULT; DRIVER, RUNTIME AND OS VERSIONS ARE FILLED FROM THE PLATFORM CONFIGURATION RECORD BEFORE SUBMISSION.

Platform	Memory	Runtime device used
AMD Ryzen AI Max+ 395 (Strix Halo) APU, Radeon 8060S iGPU	128 GB unified; firmware-reserved GPU block of 65536 MiB	iGPU via ROCm
NVIDIA RTX 4070 Laptop GPU	8188 MiB discrete VRAM	discrete GPU, layers moved to CPU above 8192 tokens
Intel Core Ultra X7 358H (Panther Lake), Arc B390 iGPU	unified; one shared Vulkan heap	CPU as shipped; iGPU when enabled

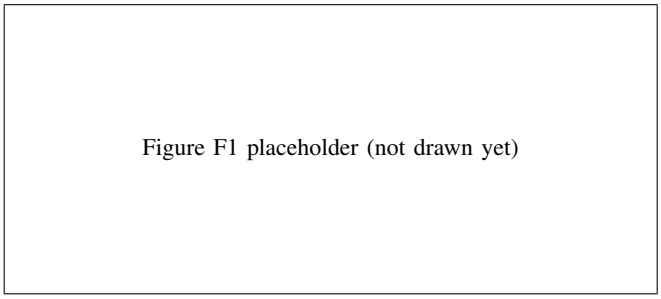


Fig. 1. Platform and memory-architecture diagram: the three platforms, where KV cache, weights and host-visible buffers live, and which runtime backend is used (drawing only).

IV. MEMORY-ARCHITECTURE BOUNDARIES

Claim. The same event, a context window whose KV cache no longer fits the memory the GPU is meant to use, has a different cost shape on each memory architecture: on the AMD APU the server keeps running past the boundary at almost unchanged speed, on the Intel iGPU the server either starts or refuses at one shared-heap limit, and on the discrete GPU the cost is expected to grow with the share of the model pushed out of VRAM. A sizing rule or a router that ignores

the architecture is wrong on at least two of the three. Fig. 2 carries this claim; the AMD and Intel panels are measured and the NVIDIA panel is not yet complete.

AMD APU: a soft spill, then a hard crash far beyond it. We ran llama-3.3-70b and qwen3-32b in llama-server with the Vulkan backend and every layer on the GPU (`-ngl 99`), stepping the window and recording per-process dedicated and shared GPU memory, decode rate and time to first token (TTFT). The platform offers three candidate boundaries: a firmware-reserved dedicated block of 65536 MiB, a Vulkan device-local heap of 76218 MiB, and a Vulkan total of 114327 MiB. We call a point a spill when shared GPU usage grows by more than 1024 MiB over the smallest window. For llama-3.3-70b the last window without spill is 65536 and the first window with spill is 100096, the window the server reported for a request of 100000. At 100096 the process held 1.122 of the dedicated block but only 0.965 of the device-local heap: the onset follows the dedicated block, not the heap the Vulkan driver advertises, so FH1 normalizes the AMD x axis by the dedicated block and says so. Crossing the onset is nearly free. The decode ratio across it is 0.983 and the TTFT ratio 1.004; decode fell from 5.284 ± 0.004 tok/s at 65536 to 5.196 ± 0.010 tok/s at 100096, and at 131072, with 18540 MiB more in shared memory than at the smallest window, it was still 5.176 ± 0.015 tok/s. Nothing in the server's response tells the client it has spilled.

The hard boundary sits much further out and is reached only through clamping. The model's trained context is 131072, and a request above it is served at that value: the last window that started was 221440, clamped to 131072, and the first that failed was 221696, with exit code 3221226505. At 221696, fresh server processes crashed with STATUS_STACK_BUFFER_OVERRUN at warm-up 5 whether the model file was read cold after a standby purge or warm from the file cache. So the AMD platform has three regimes for this model: it fits, it spills silently at almost no cost, and, for requests far above the trained context, it crashes.

qwen3-32b gives the fail side but not the spill side. The largest unclamped window without spill is 20480, and no unclamped spill point was measured, so the row has no spill cost. A request for 290000 was served at 40960 and did spill (shared +28159 MiB), at 11.190 ± 0.011 tok/s against 11.451 ± 0.005 tok/s at 20480; because that point is clamped, it is plotted but not used as a boundary. In the start-only bisection the last start was 367104, clamped to 131072, and the first failure 367360, with exit code 3221226505. The two qwen3-32b runs clamp to different values (40960 and 131072) because their launch flags differ; the panel labels each point with its launch. A dedicated qwen3-32b spill sweep between these points is planned but not yet complete, so the qwen3-32b onset is reported as "above 20480" only.

Intel iGPU: a hard wall that first-principles accounting predicts. On the Intel iGPU there is no soft regime: a model either starts or the runtime refuses it. Across five models, the refusal point is consistent with one shared-heap limit L in (48112.7, 48124.0] MiB, 247 MiB above the 47866 MiB budget the driver reports, from 92 lower and 22 upper bounds. The limit counts host-visible buffers as well as device memory: accounting for device memory alone is inconsistent, because the largest start (47945.2 MiB) exceeds the smallest refusal (47562.5 MiB). Counting weights, KV cache and compute buffers against the shared heap, with L fitted leave-one-model-out, predicts each model's flip point within -257 to -26 tokens (-0.11% to -0.01%), against -2890 to -820 tokens for model file size alone and -1092 to +968 tokens for device-only accounting. The flip points span an order of magnitude: (23296, 23552] tokens for llama-3.3-70b and (115712, 115968] for qwen3-32b. Two limits apply. These are start tests only, and for several models the flip point lies beyond the trained context, so it is a memory boundary, not a usable window. And the result is for one driver and boot state; we do not claim the 247 MiB margin transfers.

NVIDIA discrete GPU: placement measured, cost pending. On the discrete GPU the runtime itself decides how much of the model leaves VRAM. With 8188 MiB of VRAM, Ollama keeps llama3.1:8b fully on the GPU at windows of 4096 and 8192 and moves layers to the CPU above them, with VRAM shares of 0.870 at 16384 and 0.656 at 32768; for qwen3:8b the shares are 0.805 and 0.592. This places the NVIDIA points on the FH1 x axis. The y axis, decode slowdown as a function of the spilled share, comes from an earlier sweep with llama-server driving the driver's system-memory fallback directly; that sweep is not yet part of our verified results, and the forced-failure point with the fallback disabled needs a run on the discrete-GPU laptop that has not been done. The NVIDIA panel is therefore pending, and the two overflow paths (the runtime moving layers to the CPU, and the driver spilling allocations into system memory) will be labelled separately when it lands.

Co-running work on unified memory. On both unified-memory platforms the GPU shares memory bandwidth with the CPU, so a CPU-side workload is part of the boundary. On the Intel platform a co-runner raised llama-3.3-70b's TTFT by a factor of 1.42x. On the AMD APU the largest TTFT ratio under any co-runner was 1.129x, and the decode rate separates a bandwidth hog from a compute hog (0.905x-0.928x against 0.987x-1.000x of the no-load rate), while package power for the two large models stayed flat at about 83.6 W across every condition. The Intel platform has no bandwidth-hog run, so the two vendors are not yet compared on the same metric.

V. Runtime Policy

Claim. The window an agent actually gets is set by the runtime's policy for the device it detects, not by the model or the memory, and when a prompt outgrows that window the runtime reports success and silently keeps half of it.

Default windows follow the detected device. Ollama's Vulkan backend drops both integrated GPUs by the same policy, but Ollama also discovers the AMD iGPU through ROCm, with no opt-out, so the AMD APU runs on its iGPU while the Intel laptop, which has no ROCm path, falls through to the CPU. The default window follows. On the Intel laptop

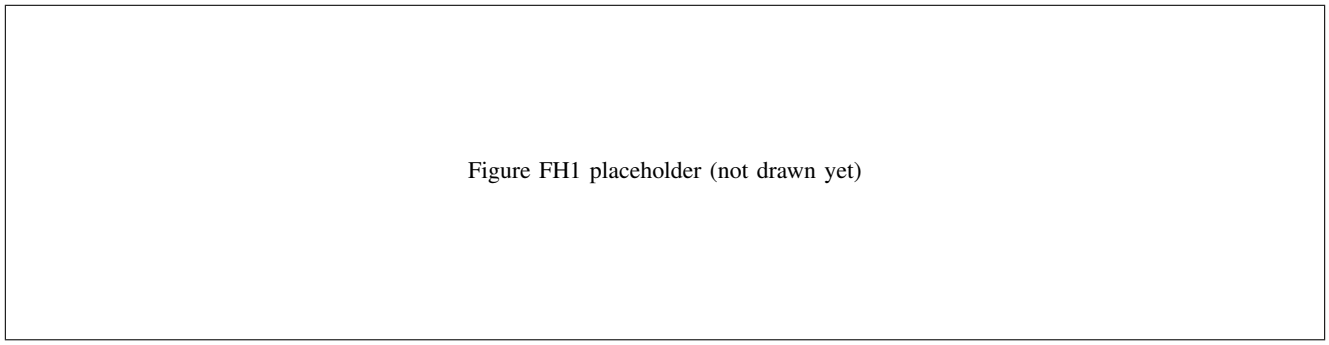


Fig. 2. Three panels, shared x axis = requested or served window as a fraction of the platform's boundary (AMD: the 65536 MiB dedicated block, stated in the caption; Intel: the shared-heap limit; NVIDIA: VRAM), y axis = cost of crossing it (decode and TTFT ratio; for Intel, start or refuse). Clamped points are drawn but marked; each point is labelled with its launch and its overflow path. Measurement note for the caption: each AMD point is one call per fresh server start without a warm-up call (5 or 3 calls per point), which departs from this paper's 1 warm-up + N convention. If space is short, FH1 replaces F2 and F3.

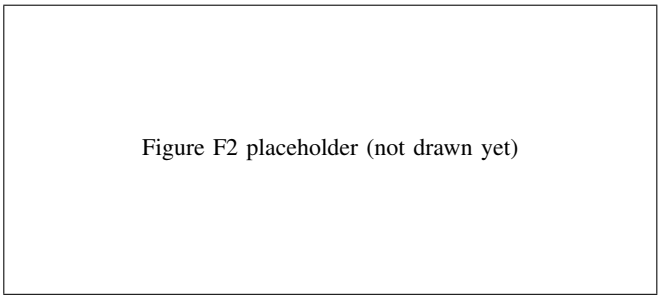


Fig. 3. Intel flip point: measured (last start, first refusal] per model against accountings A (file size), B (device only) and C (shared heap), all five models.

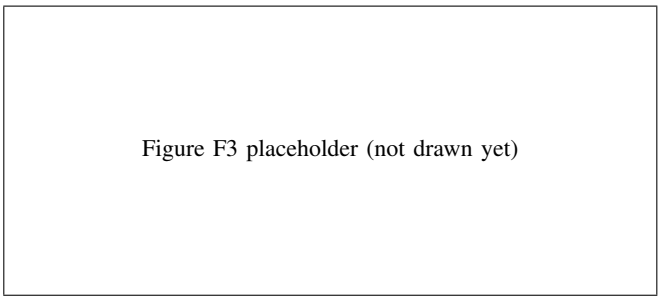


Fig. 4. NVIDIA spill onset: TTFT and decode slowdown against window with the spilled share, plus the forced-failure point; pending (the spill sweep is not yet verified and the forced-failure run has not been done).

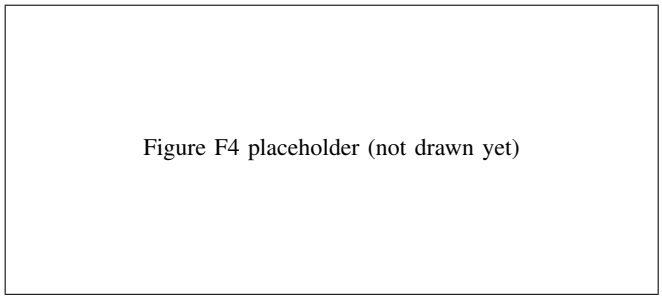


Fig. 5. Co-runner effects by vendor: TTFT and decode ratio per condition, Intel against AMD; Intel decode under a bandwidth hog is missing. Intel CPU co-runner TTFT ratio per model 1.417x to 1.437x, mean 1.429x (+42.9%) over six models; Intel qwen3-32b TTFT dose response by co-runner placement e2:+2.0%, lp4:+3.2%, e4_clusterA:+5.7%, e4_clusterB:+5.3%, e4_split:+6.0%, e6:+14.3%, e8:+30.2%, e8_lp4:+41.4%; AMD bandwidth-hog against compute-hog TTFT gap 2.1-2.6% across 5 models; AMD decode ratios under a bandwidth hog (B4) 0.905x-0.928x and a compute hog (S4) 0.987x-1.000x of the no-load rate; AMD largest TTFT ratio 1.129x. Note: the Intel co-runner ratios, the Intel dose response and the AMD TTFT gap are recomputed from the raw result files and replace the rounded values of earlier drafts.

TABLE II
DEFAULT CONTEXT WINDOW PER PLATFORM FOR QWEN3-4B-2507 / LLAMA3.1:8B / QWEN3:8B, ONE RUNTIME (OLLAMA).

platform (device used)	default window, three models	over-length prompt at the default
AMD APU (iGPU via ROCm)	262144 / 131072 / 40960	success, half kept
Intel laptop (CPU)	4096 / 4096 / 4096	success, half kept
NVIDIA laptop (discrete GPU)	not measured / 4096 / 4096	HTTP 200, evaluated 2050

the runtime loaded 4096 tokens for each of three models, while on the AMD APU it loaded 262144 for qwen3-4b-2507, 131072 for llama3.1:8b and 40960 for qwen3:8b, in each case the model's native window (capped only for qwen3:8b). The discrete GPU also got 4096 for both models we could measure there. The runtime's own log reports the rule behind the two small values: a default sized from detected VRAM, 4096 for the discrete GPU's 8.0 GiB and 32768 for the Intel iGPU's 36.2 GiB when that iGPU is enabled. Table II lists the values. The spread from 4096 to 262144 for the same weights comes from device detection, not from capacity: the Intel laptop has 36.2 GiB visible to the runtime and still gets 4096 when its iGPU is dropped.

The default decides the outcome. On the Intel laptop we ran the same agent sessions twice, once as shipped on the CPU and once with the iGPU enabled for the job's own server process. On the CPU both models loaded a 4096-token window, crossed it at turn [3], and failed silently in 5/5 sessions each, every failure after the window was exceeded. On the iGPU the same models loaded 32768, crossed it only at turn [23] and [20, 21], and no failure came after the window: 0/5 for llama3.1:8b and, for qwen3:14b, 1/5 of baseline type.

The pre-registered kill criterion was not triggered on either backend. This contrast changes backend and window together, so it shows that the default decides whether a session fails, not that the window alone does; Section VI-A isolates the window.

Overflow is silent. When a prompt exceeds the loaded window, the runtime returns success and processes num_ctx/2 + 2 tokens, in all 4 checked cases (4096→2050, 8192→4098, 32768→16386, 40960→20482), through its own `--context-shift --keep 4` server flag. On the discrete GPU, prompts of 8192, 16384 and 32768 tokens sent to a 4096 window each returned HTTP 200 with 2050 tokens evaluated, for both models. No error, warning or response field tells the client.

Changing the window, and what persists. A user can override the default in three ways, and they differ in how long the override lasts. On the AMD APU, whose baseline for llama3.1:8b is 131072, a per-request `num_ctx` of 12288 was applied but reverted to 131072 after a restart; a Modelfile `PARAMETER num_ctx 10240` on a derived tag persisted across a restart while the base tag stayed at 131072; and `OLLAMA_CONTEXT_LENGTH` of 6144 applied while the variable was set and reverted to 131072 when the server restarted without it. Only the derived tag survives a restart without the client's help. This was measured on the AMD APU only; the other two platforms have not run it.

Within the window, KV precision is the lever. We pre-registered a comparison of eight context policies on the Pareto frontier of rules, history, time and cost (Section VII gives the router verdict). On the AMD APU every policy kept every rule, and the frontier held one KV-cache policy per model. For llama3.1:8b a q4_0 KV cache took 6.6 min per session at history 100.0%, against 43.7 min at 95.8% for the runtime default; for qwen3:14b a q8_0 cache took 75.4 min against 214.8 min, at the same history recall of 62.5%. Our quantized-cache policies also turn on flash attention, so we ran a control with flash attention on and the cache at f16: 43.4 min for llama3.1:8b and 206.6 min for qwen3:14b, within tolerance of the default, so the speedup is KV precision, not flash attention. The first f16 recall measurement for llama3.1:8b had an errored call; a rerun found 11/24 needles at f16 against 13/24 at q8_0 (Fisher p = 0.773) and 14/24 at q4_0 (p = 0.564), no difference shown, with needles within a call not independent. All of this is on the unconstrained AMD APU, where the default is the native window. The simulated-constraint version, with the window capped at 8192 and 4096, is not yet complete, and the constrained platforms have not run this comparison.

Install path and first-call stall. Two further runtime choices change behaviour without changing the weights. Installing qwen3-4b-2507 from a bare model file instead of with the library template changed native tool calls on the first call from 11/30 to 2/30, while task-tool correctness stayed at 5/30: behaviour follows the install path, and neither path passes our task-tool gate. And the first request on a new prompt waits while the previous request's KV state is moved: on the AMD APU 658/919 first calls were more than 1.5x slower than the repeat, with a stall median of 15.7-40.8 s, a runtime cost that a first-call TTFT measurement mistakes for a model property.

Figure F5 placeholder (not drawn yet)

Fig. 6. Default window per platform and model over the CDF of agent step lengths in public traces.

VI. Agent Consequences

A. The Window Result

A default the user never sees. On the Intel laptop, with no window requested, the same Ollama and the same two models loaded 4096 tokens on the CPU and 32768 on the iGPU. On the CPU every session crossed the window at turn [3] and then failed silently, 5/5 for llama3.1:8b and 5/5 for qwen3:14b. On the iGPU no failure came after the window (0/5 and 1/5 baseline-type). The CPU is the Intel default; the same tiering gives llama3.1:8b 4096 on Intel, 131072 on the AMD APU and 4096 on the NVIDIA laptop. This contrast changes backend and window together, so it motivates the controlled result rather than replacing it.

The controlled result. On the AMD APU we ran three models at the Ollama default and at 32768, 16384, 8192 and 4096, five seeds per cell: 75 sessions and 3000 turns, none surfacing an error before its first failure. After-window silent failures rise as the window shrinks: 0/15 at 32768, 2/15 at 16384 (excess over default +0.133, 95% CI +0.000 to +0.267), 8/15 at 8192 (+0.533, 95% CI +0.333 to +0.667) and 10/15 at 4096 (+0.667, 95% CI +0.533 to +0.800), over a default tier with 0/15. The CI excludes zero at 8192 and 4096. Rule failures, as distinct from recall misses, appear only at 8192 and 4096, and at 4096 no session of any model kept its history to the first canary turn (intact 0/0/0/0/0/0/0/0 of 5 for each model).

The default tier's gated count is 3/15, all llama3.1:8b recall misses inside the window. The pre-registered kill criterion, counting every metric, finds 10/15 silent sessions there, all baseline-type: not killed (killed=False), but none of it is context loss.

Models differ (Table III). llama3.1:8b already fails at 16384, 2/5, and 5/5 at 4096, excess +1.000. qwen3:14b has no after-window failure above 4096, and its 4096 sessions lapse on different rules: the first gated failures are rule 5 or rule 3 at turns 5 to 30, while rule 2 held on 40/40 turns in every seed; we name the failing rule per model rather than say rules broke. qwen3:8b reversed the ordering at five seeds; a pre-registered extension to 10 seeds per tier tested whether that was noise. Pooled, qwen3:8b failed silently in 6/10 sessions at 8192 and

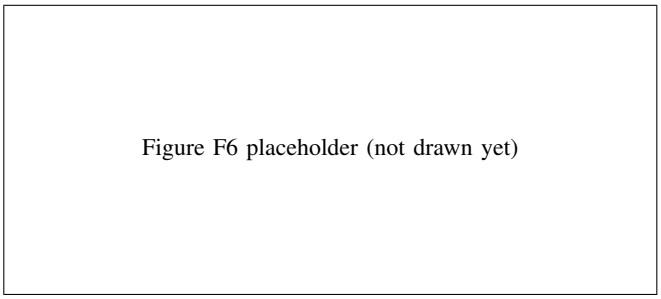


Fig. 7. Per-tier session survival curve (intact sessions against turn), with the after-window excess per tier and its 95% CI as a second panel.

2/10 at 4096 (Fisher exact two-sided p = 0.170), which the pre-registered rule reads as resolved as noise. The direction persisted: the new seeds alone gave 2/5 at 8192 and 1/5 at 4096, and the excess over default is +0.600 (95% CI +0.300 to +0.900) at 8192 but +0.200 (95% CI +0.000 to +0.500) at 4096, where no excess is shown. We report the cell as within noise and do not claim that 4096 is safe for qwen3:8b.

Protocol check: tools on every call. A pre-registered rerun offers tools on every call, on a different task workload, with five seeds per tier and a `num_keep` tier (Section VII). It rests on qwen3:14b alone. llama3.1:8b failed the task validation, and qwen3:8b, added by a later pre-registration with no waiver, failed it too: it called the task tool correctly on 61/81 baseline turns and reached the task end state on 65/90 (72.2%), so its real run refused to start, as designed. For qwen3:14b the pre-registered reading was that the effect holds at 4096 if the excess CI lies above zero. Within this run it held: 5/5 against 0/5 at the default, excess +1.000 (95% CI +1.000 to +1.000). Wrong task end states were 11.3% at the default, 12.7% at 16384, 35.3% at 8192 and 42.0% at 4096, every wrong turn silent: excess +24.0 points (95% CI +18.7 to +29.3) at 8192 and +30.7 points (95% CI +26.0 to +34.7) at 4096, none shown at 16384. A deviation was declared before the data: qwen3:14b reached the task end state on 80/90 validation turns, below the bar, so absolute wrong rates include baseline errors.

Scope: scale and family. The pre-registered H-scale hypothesis stated that both R2 findings, rule loss at 4096 and history loss with rules intact at 32768, hold from 30B to 70B, with three seeds per cell. qwen3:32b lost rules at 4096 in 2/3 sessions but showed history loss with rules intact in only 1/3 at 32768 (mixed); at its default it loaded 40960 tokens and crossed the window late, at turns 24, 25 and 26. llama3.3:70b lost no rules at 4096 (0/3, every session over the window) and lost history with rules intact in 3/3 at 32768, the pre-registered alternative "resists rule loss". Overall the verdict is mixed: H-scale is not supported. mistral-small3.2:24b, from another family, also resisted rule loss (0/3 at 4096, history loss 2/3 at 32768). Within what we tested: Rule loss appears in the 8B and 14B models (5 seeds) and in qwen3:32b (2 of 3 sessions at 4096), and not in llama3.3:70b or mistral-small3.2:24b (0 of 3 each). History loss appears in all of them. We do not claim rule loss in general.

B. The Mechanism on AMD and NVIDIA

What the runtime removes. We reran llama3.1:8b on the AMD APU, one session per tier (80 calls each), with Ollama in debug mode, its log sliced per call, and a render-only copy of each request. The rendered prompt matched the runtime's count on every check (9/9 within 1% at 4096; render/log disagreements 0). Three mechanisms appear. First, Ollama drops whole messages from the front once the conversation outgrows the window, on 54/80 calls at 16384, 68/80 at 8192 and 76/80 at 4096, always keeping the system message ("system kept on all True") and usually cutting mid-turn. This loses history but keeps the rules in view. At 4096 two lower-level steps then remove the system prompt. llama.cpp's context shift fired on 34 calls from turn 4, each keeping n_keep [5] tokens and discarding the next [2045], 29 of them on the final-answer call; and once Ollama's own token cut reduced a prompt of 4097 tokens to 2050, keeping 4. That is the runtime's overflow rule, which returns success and keeps num_ctx/2 + 2 tokens (4096→2050). No step raised an error (exceed-context errors 0). The report's working hypothesis, "drop old turns, keep the system message", is therefore refuted over the citable tiers by the 4096 token cut (verdict refuted) and confirmed tier by tier at 32768, 16384 and 8192.

NVIDIA replicates the first two steps. One session at 4096 on the NVIDIA laptop passed the same render check (9/9 within 1%). Ollama trimmed whole messages on 75/80 calls from turn 3 and kept the system message on every one, and the context shift fired 33 times; no token cut occurred and no error surfaced. That row lacks n_keep, so the claim that the shift discards the system prompt is AMD-only; whether the token cut occurs on NVIDIA is open.

A second silent difference: tool calls as text. In the NVIDIA validation, llama3.1:8b's negative control at 32768, where nothing is truncated, missed canaries in two of five sessions (seeds 20260903 and 20260905, 2 misses each): the final answer was a tool call written as plain text. By the gate set in advance, this held back the NVIDIA real runs. Three pre-registered checks on the AMD APU kept model files, seeds and settings and moved one factor toward the NVIDIA configuration each. At 32768 on AMD's ROCm backend there were 0 of 20 misses, so the window does not reproduce it ("backend or offload effect"). Forcing NVIDIA's CPU offload split on AMD, 22 of 33 layers on the GPU with VRAM share 0.656, again gave 0 of 20 ("backend effect"). Switching only the backend, to Vulkan on the same AMD GPU and verified in 5/5 sessions, gave 2 of 20 misses, at seed 20260903, turn 5, as a text call to `lookup_fact`: "backend implementation, not the vendor". The failure follows the llama.cpp backend implementation (CUDA and Vulkan, not ROCm, in these seeds), not the GPU vendor: one fragile decision point, turn 5 of seed 20260903 on Vulkan, in the seed that also missed on CUDA, separates a correct agent from a silent miss. Five seeds cannot show a zero is impossible, and the kernel is not identified; identical model files and settings still yield a different agent on a different backend, with no error.

TABLE III

GATED SILENT FAILURES PER TIER AND MODEL ON THE AMD APU. SILENT / AFTER WINDOW / BASELINE-TYPE GATED FAILURES, OF 5 SESSIONS PER CELL; INTACT = SESSIONS WITH NO GATED FAILURE AND NO CANARY MISS AT TURNS 5/10/15/20/25/30/35/40. THE QWEN3:14B 4096 CELL COUNTS LAPSES OF RULE 3 AND RULE 5, NOT RULE 2. THE QWEN3:8B 8192 AND 4096 CELLS ARE FIVE SEEDS HERE; THE TEXT QUOTES THE 10-SEED POOLED CELL.

tier	llama3.1:8b	qwen3:14b	qwen3:8b
Ollama default	3 / 0 / 3; intact 4/4/4/2/2/2/1/1	0 / 0 / 0; intact 5/5/5/5/5/0/0/0	0 / 0 / 0; intact 5/5/5/4/4/0/0/0
32768	1 / 0 / 1; intact 5/5/5/3/3/0/0/0	0 / 0 / 0; intact 5/5/5/5/5/0/0/0	0 / 0 / 0; intact 5/5/5/4/0/0/0/0
16384	2 / 2 / 0; intact 5/5/0/0/0/0/0/0	0 / 0 / 0; intact 5/5/0/0/0/0/0/0	0 / 0 / 0; intact 5/5/0/0/0/0/0/0
8192	4 / 4 / 0; intact 5/0/0/0/0/0/0/0	0 / 0 / 0; intact 5/0/0/0/0/0/0/0	4 / 4 / 0; intact 3/0/0/0/0/0/0/0
4096	5 / 5 / 0; intact 0/0/0/0/0/0/0/0	4 / 4 / 0; intact 0/0/0/0/0/0/0/0	1 / 1 / 0; intact 0/0/0/0/0/0/0/0

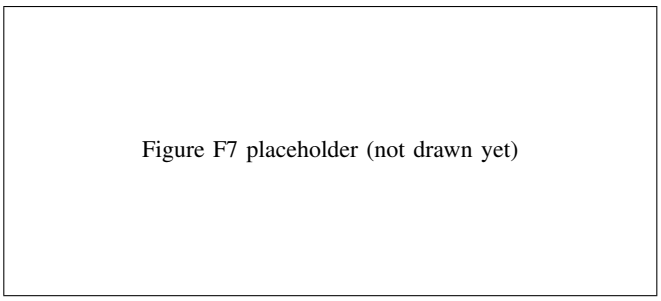


Fig. 8. Per-turn event raster per tier (message drops, context shifts, token cut), one row per tier, with the NVIDIA 4096 session as an extra row next to the AMD 4096 row.

How the window is set, and whether it sticks. On the AMD APU, where llama3.1:8b defaults to 131072, a per-request `num_ctx` of 12288 loaded 12288 but a plain request after a restart was back at 131072; a Modelfile `PARAMETER num_ctx` of 10240 on a derived tag loaded 10240 and survived the restart; and `OLLAMA_CONTEXT_LENGTH` of 6144 applied while set and reverted once the server restarted without it. Only a derived tag persists: a client that omits the per-request option silently gets the default back. This check has not yet run on NVIDIA or Intel.

VII. FIXES

Client-side trimming splits by model, on two vendors. A client that never sends an over-length prompt should keep the runtime from cutting the system prompt. We pre-registered that fix: before every call the client renders and counts the request with the model's tokenizer and drops whole old turns until the system prompt, the kept turns and the answer budget fit the window with a 5% margin, always keeping the system prompt. A model *recovers* when every rule in use is at or above 90%, and the hypothesis needed both models to recover at 4096. Mechanically it worked: 0 context shifts, 0 token cuts, 0 Ollama message drops and 0 exceed-context errors in every cell. The verdict is *refuted*. On the AMD APU llama3.1:8b recovered at 8192 and 4096; qwen3:14b recovered at 8192 but not at 4096, where its `log_event` rule fell to 61.7%, failing first at turn 3 in every seed (3/3/3). On the Intel laptop's CPU at 4096 the split replicates: llama3.1:8b recovered, qwen3:14b did not, again on `log_event`, overall refuted. The cause is the fix itself: untrimmed, qwen3:14b kept rule 2 on 40/40 turns in every seed. On the first failing trimmed turn its first call requested only `lookup_fact`, where the untrimmed run called `log_event` and then `lookup_fact`: at 4096 the budget leaves room for only 1-2 turns besides the system prompt, so trimming removes the in-context examples of the two-call pattern the model imitates. Naive trimming swaps silent context loss for deliberate history loss and, for this model, a new rule failure.

num_keep fixes one model's rules, on the token-cut path. The request option `num_keep` sets how many leading tokens the context shift and the token cut keep. It matters only where a shift or a cut happens; a whole-message drop keeps the system message anyway (Section VI-B). We pre-registered a test for llama3.1:8b at 4096, the path where both steps fired, with `num_keep` set to the system prompt's measured token length (601, 594 and 597 per seed). All 105 context shifts kept n_keep [595, 598, 602]. The rules in use held (rule 1 at 94.2%, rules 3 and 4 at 100.0%), which the pre-registered rule reads as "one-line fix". It fixes the rules and nothing else: Ollama still dropped messages on 224 of 239 calls and the history canary was missed on 24 of 24 checks. Nor does it remove the window effect in the protocol check: within that run the `num_keep` tier at 4096 lowered qwen3:14b's wrong-task rate from 42.0% to 30.0%, but its excess over the default stayed at +18.7 points (95% CI +16.0 to +21.3) and its gated excess at +1.000; the pre-registered reading is that `num_keep` does not remove it. In the R2 harness a num_keep run for qwen3:14b at 4096 reproduced the unflagged run turn for turn (identical final text 120/120), so for this model the flag changed nothing there, and what lowered the task-workload rate is not established.

A router does not dominate. We pre-registered a comparison of eight context policies (runtime default, largest window that starts, client trimming, a router escalating to a cloud model, an auto-sized window, `num_keep`, q8_0 and q4_0 KV caches), with the hypothesis that the router is on the Pareto frontier of rules, history, time and cost on every platform. It is *refuted*: on the AMD APU the router was dominated for both models. Every policy kept every rule (rule 100.0% in every cell), and the frontier held one KV-cache policy per model. For llama3.1:8b, q4_0 reached history 100.0% in 6.6 min per session against the router's 95.8% in 43.5 min. For qwen3:14b, q8_0 cut session time from 214.8 min at the runtime default to 75.4 min at the same history recall, 62.5%, while the router took 135.1 min and USD 0.8419 per session for no gain. The refutation holds on the unconstrained AMD platform,

whose default window is the model's native one, so no policy there faced the small-window regime where loss occurs; the constrained platforms are pending.

Recovering after a silent failure is costly and incomplete. The fallback for a session that has already failed is to hand the whole history to a cloud model late and let it finish the task. We measured that on every wrong task of the protocol-check task run (Section VI-A): a cloud model with the full client-side history fixed 167/197 (84.8%) of them, 95% Wilson CI [79.1%, 89.1%], at a mean of USD 0.0227 per task. Matched on the same task and seed, the late redo cost 1.76x a fresh run of that task from the start (mean USD 0.0226 against USD 0.0129 over 119 matched rows). It also cannot undo what the local model already did: of the 30 tasks still wrong, 28 needed a repair of state the local model's earlier actions had changed, and 2 were cloud-model errors. An agent's tool calls are executed, not drafted, so a late redo inherits them. On the Intel CPU at its 4096 default (projected from the AMD run at the same window), escalating every step the window check flags reaches the cloud model's accuracy but costs more than running every session in the cloud from the start. The alternative is to decide at session start, per device and workload, whether a whole session runs locally with the recommended setting or goes to the cloud. With the rule "run locally if local accuracy at the recommended setting is within a tolerance of the cloud model's", no measured device qualifies at matched quality: the AMD APU at its best setting is 7.8 points below the small cloud model and 11.1 below the mid-tier one, and the Intel iGPU, measured by session pass rate rather than task accuracy, is 16.7 and 20.0 points below. At matched quality the measured answer is therefore the cloud from the start. Local execution pays only where a user accepts a stated accuracy gap, or where configuration closes it, which the per-device settings of Sections IV and V address and the policy runs on the constrained platforms will test. Step-level escalation stays a narrow middle case, since window overflow does not pick out the wrong steps (Section VII-A).

A. Predictor

A context shift is predictable before the call. The step that discards the system prompt at the smallest window is llama.cpp's context shift (Section VI-B), and its trigger can be read from the llama.cpp source rather than fitted: a shift happens on a call when the prompt left after the message-level trim, plus the tokens generated, exceeds the loaded window. We scored that rule on the AMD mechanism sessions, one per tier, against the shifts the server log recorded. At 4096 it gave TP 34 FP 0 FN 0 TN 46, and at 32768 TP 10 FP 0 FN 0 TN 70, precision and recall 100.0% in both, with misses 0 over all tiers. The exact form matters, which is evidence that the rule is the mechanism and not a correlate: P+C>n_ctx gives TP 44 FP 0 FN 0, while the prompt alone, P≥n_ctx-1, gives TP 0 FP 0 FN 44. The generated count is not known before the call, so a client substitutes the generation budget: P+num_predict>n_ctx gives TP 44 FP 22 FN 0, a before-the-call flag that misses no shift at the cost of false alarms. The rule has no tunable parameter, but it was built and scored on the same sessions; no held-out observed sessions exist yet.

Figure F8 placeholder (not drawn yet)

Fig. 9. Rule compliance per rule in use at 4096 and 8192, mitigation against the real run, per model. The qwen3:14b 4096 panel shows that rule 2 falls only with trimming (40/40 untrimmed against 24/40, 25/40, 25/40 trimmed).

It predicts the shift, not the failure. Applied to the real run as a prediction only, where no server log exists, the rule flags 102/240 calls for llama3.1:8b at 4096 and 0/240 for qwen3:14b, yet qwen3:14b also loses rules at 4096. Its lapses there come with no predicted shift, consistent with the reconciliation in Section VII: for that model the loss runs through what the message-level trim removes, not through the shift. What the trim leaves at the front of the window is usually a fragment: at 4096, 61/76 (80.3%) of truncated calls began with an orphaned message.

Window overflow is not a general wrong-answer detector. A tempting use of the flag is to escalate exactly the steps that overflowed. We measured it as a detector of wrong task outcomes on the protocol-check task run of Section VI-A (qwen3:14b, AMD APU), with the predictor "this task turn's prompt exceeded the loaded window" and the outcome "the task ended wrong". At the runtime default no turn overflowed, and the flag missed 17/17 wrong tasks (recall 0.0%). At 16384 it reached precision 15.4% and recall 21.1%; at 8192 precision 45.3% and recall 90.6%; at 4096 recall 100.0% but precision 42.9%, flagging 147 of 150 turns (TP 63 FP 84). Where the window is large, wrong answers have other causes that the flag cannot see; where it is small, nearly every step overflows and the flag no longer separates good steps from bad. The predictor is therefore a tool for explaining and anticipating system-prompt loss, not a cheap filter for which steps to send to a cloud model.

VIII. Threats to Validity

Platforms and versions. We measure one machine per vendor, so a platform effect cannot be separated from that machine's driver and memory configuration. Agent runs use Ollama 0.34.4 and direct runs llama.cpp b10970; default windows and trimming are runtime code and can change in any release. On the Intel laptop, CPU and iGPU differ in backend and loaded window together (4096 against 32768), so the motivating contrast is not a controlled test of either. The NVIDIA laptop contributes validation and mechanism sessions only, because its validation gate held back the real agent runs,

and the router and context-policy comparison ran on the AMD APU alone, whose default window is the model's native one.

Statistics. Cells have three to five seeds, ten for qwen3:8b. A 0/5 or 5/5 cell has a point bootstrap interval that understates uncertainty, and the scale and family results rest on three sessions per cell. qwen3:8b's lower failure count at 4096 than at 8192 is within noise ($p = 0.170$), and we claim nothing from it. Sampling uses temperature 0 and a fixed request seed, so seeds vary the session script, not the sampling; nonzero temperatures are untested. Where a script times or scores repeated calls it follows one warm-up call plus N measured calls ($N = 3$ for the KV-precision recall check, whose needles within a call are not independent, so that check is descriptive); session-time comparisons are over three seeds.

Workload. The R2 rules are synthetic and checkable; deployed system prompts differ. Guardrail rules 6 to 8 (privacy, refusal, confirmation) already fail at full context for both models tested, for example rule 6 at 0.417 and 0.250 of bait turns, so they are excluded from every window measurement and support no claim here. The tools-on-every-call task workload rests on qwen3:14b alone, under a deviation declared before the data: its task end state was 80/90 at validation, below the 90% bar, and we proceeded with that model only. qwen3:8b (65/90) and llama3.1:8b (72/90) were refused by their gates. A second model, mistral-small3.2:24b, is in validation for this workload and has no result yet.

Cost figures. Cloud cost per real agent task is priced from traces, not executed, and its caching case is an assumption. The hybrid comparison prices an escalated task at the measured full-history redo cost for the mid-tier cloud model, and at the small model's all-cloud cost per task, an interim value, because the redo ran on the mid-tier model only. Our first cloud redo of wrong local tasks was a broken protocol: 109 of its 118 failures were turns where the model's default thinking spent the whole 384-token output budget and returned an empty answer, so its fix rate measured the token budget, not the cloud model. We cite no fix rate or cost from it; the corrected full-history redo (thinking limited to between tool calls, 2048 output tokens per call against the local run's 384) is the one reported in Section VII.

Provenance. Runs before 2026-10-14 did not record the harness file's hash ("harness blob not recorded"). Their identity rests on reruns: the same seeds gave identical final text on 120/120 turns in runs days apart.

IX. Conclusion

Local LLM agents lose context without telling anyone. On every platform we measured, a conversation that outgrows the window is trimmed, shifted or cut, and the request still succeeds. Rule loss appears in the 8B and 14B models (5 seeds) and in qwen3:32b (2 of 3 sessions at 4096), and not in llama3.3:70b or mistral-small3.2:24b (0 of 3 each). History loss appears in all of them. Rule loss depends on the model as well as the window, and we do not claim it in general. The window is often not the user's choice: the runtime picks it from the detected device, and an unmodified install on an Intel CPU ran agents at 4096 tokens. That is the window at which, on the AMD APU, qwen3:14b ended 42.0% of tasks in a wrong state against 11.3% at its default, all silently. Running locally removes the cloud bill, but the wrong answers it adds carry no signal, and the one signal a runtime could raise, window overflow, misses every wrong answer produced inside the window.

For practitioners, five lessons follow. Set the window explicitly for each device and verify the loaded value, because a per-request setting does not survive a restart and any client that omits it silently gets the default back. Log when a prompt exceeds the loaded window, since nothing else will, but do not treat that log as a correctness check. Try a quantized KV cache before a smaller window: on the AMD APU it cut session time by 85% and 65% with no measured loss of rules or recall. Treat trimming as a change to the prompt rather than a neutral fix: dropping old turns also drops the examples the model imitates, and `num_keep` protects the prompt prefix only where the runtime shifts or cuts tokens. Finally, validate an agent on the backend it will run on, because identical model files and settings behaved differently on CUDA, Vulkan and ROCm.

Future work follows from what is pending. The router and context-policy comparison, refuted on the unconstrained AMD platform, must run on the Intel laptop, whose default window is small enough that loss occurs, and on the NVIDIA laptop. A hybrid local and cloud deployment needs a general verifier that catches wrong answers produced inside the window; we measured only the window flag; a late cloud redo with the full history fixed 84.8% of wrong tasks at about twice the cost of a fresh cloud run and could not undo executed actions. More models per size and family, more seeds, further runtime versions and guardrail compliance at full context would turn the model-dependence of rule loss from an observation into a predictable property.

References

[1] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," TACL (arXiv:2307.03172), 2024, TO VERIFY.

[2] C.-P. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" COLM, 2024, TO VERIFY.

[3] Y. Bai *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," ACL, 2024, TO VERIFY.

[4] P. Laban *et al.*, "LLMs get lost in multi-turn conversation," arXiv, 2025, TO VERIFY.

[5] J. Zhou *et al.*, "Instruction-following evaluation for large language models," arXiv, 2023, TO VERIFY.

[6] N. Mu *et al.*, "Can LLMs follow simple rules?" arXiv (RuLES), 2023, TO VERIFY.

[7] G. Xiao *et al.*, "Efficient streaming language models with attention sinks," ICLR, 2024, TO VERIFY.

[8] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," NeurIPS, 2023, TO VERIFY.

[9] Y. Li *et al.*, "SnapKV: LLM knows what you are looking for before generation," NeurIPS, 2024, TO VERIFY.

[10] Z. Liu *et al.*, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," ICML, 2024, TO VERIFY.

[11] C. Hooper *et al.*, "KVQuant: Towards 10 million context length LLM inference with KV cache quantization," NeurIPS, 2024, TO VERIFY.

[12] H. Jiang *et al.*, "LLMLingua: Compressing prompts for accelerated inference of large language models," EMNLP, 2023, TO VERIFY.

[13] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," ICLR, 2024, TO VERIFY.

[14] S. Yao *et al.*, "tau-bench: A benchmark for tool-agent-user interaction in real-world domains," arXiv, 2024, TO VERIFY.
[15] S. G. Patil *et al.*, "Gorilla: Large language model connected with massive APIs," NeurIPS, 2024, TO VERIFY; the Berkeley Function Calling Leaderboard may be the better citation.
[16] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" ICLR, 2024, TO VERIFY.
[17] LMSYS Org, "Routellm," GitHub repository, 2024, accessed 2026-07-23; representative query-level LLM routing implementation. [Online]. Available: https://github.com/lm-sys/RouteLLM
[18] D. Ding *et al.*, "Hybrid LLM: Cost-efficient and quality-aware query routing," ICLR, 2024, TO VERIFY.
[19] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv, 2023, TO VERIFY (TMLR 2024?).
[20] Y. Weng *et al.*, "Large language models are better reasoners with self-verification," Findings of EMNLP, 2023, TO VERIFY.
[21] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," EMNLP, 2023, TO VERIFY.
[22] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," SOSP, 2023, TO VERIFY.
[23] S. Laskaridis *et al.*, "MELTing point: Mobile evaluation of language transformers," MobiCom, 2024, TO VERIFY.
[24] Y. Song *et al.*, "PowerInfer: Fast large language model serving with a consumer-grade GPU," SOSP, 2024, TO VERIFY.
[25] A. Saxena *et al.*, "Llm in a flash: Efficient large language model inference with limited memory," 2023, representative edge/resource-constrained LLM serving perspective.